```
clc;
clear;
close all;
```

INPUTS

```
b = [0.0048 0.0193 0.0289 0.0193 0.0048];
a = [1 -2.3695 2.3140 -1.0547 0.1874];

Bc = 8;       % Coefficient word length
Bp = 16;      % Product word length
N = 200;
x = [1 zeros(1,N-1)];
```

1. COEFFICIENT QUANTIZATION

```
S = 2^(Bc-1)-1;

bq = round(b*S)/S;
aq = round(a*S)/S;

CoeffError = [b-bq a-aq];
```

2. IDEAL AND COEFFICIENT-QUANTIZED RESPONSE

```
y_ideal = filter(b,a,x);
y_coeff = filter(bq,aq,x);

e_coeff = y_ideal-y_coeff;
MaxCoeffError = max(abs(e_coeff));

SQNR_coeff = 10*log10( ...
    sum(y_ideal.^2)/max(sum(e_coeff.^2),eps));
```

3. PRODUCT QUANTIZATION

```
Sp = 2^(Bp-1)-1;

y_prod = zeros(1,N);
v = zeros(1,4);

for k = 1:N
    p = bq*x(k);
    fb = -aq(2:5).*v;

    % Quantize products to the specified product precision
    p = round([p fb]*Sp)/Sp;
```

```
    y_prod(k) = sum(p);

    % Update feedback using quantized output
    v = [y_prod(k) v(1:3)];
end

e_prod = y_coeff-y_prod;
MaxProdError = max(abs(e_prod));

SQNR_prod = 10*log10( ...
    sum(y_coeff.^2)/max(sum(e_prod.^2),eps));
```

4. ZERO-INPUT LIMIT-CYCLE TEST

```
state = [0.01 0.01 0.01 0.01];
y_zero = zeros(1,N);

for k = 1:N
    acc = -aq(2:5)*state.';
    y_zero(k) = round(acc*Sp)/Sp;

    state = [y_zero(k) state(1:3)];
end

LimitAmplitude = max(abs(y_zero));
LimitSamples = find(abs(y_zero)>1e-12);
```

5. OVERFLOW / SATURATION TEST

```
xo = 1.2*ones(1,N);
y_over = filter(bq,aq,xo);

% Normalized saturation
y_sat = min(max(y_over,-1),1);

OverflowCount = sum(abs(y_over)>1);
MaxOverflow = max(abs(y_over));
```

6. OUTPUT RESULTS

```
fprintf('\n========== QUANTIZATION RESULTS ==========\n');
fprintf('Coefficient word length = %d bits\n',Bc);
fprintf('Product word length = %d bits\n',Bp);

fprintf('Maximum coefficient/output error = %.6e\n', ...
    MaxCoeffError);
fprintf('Coefficient SQNR = %.2f dB\n',SQNR_coeff);

fprintf('Maximum product quantization error = %.6e\n', ...
    MaxProdError);
fprintf('Product SQNR = %.2f dB\n',SQNR_prod);
```

```
fprintf('Zero-input maximum amplitude = %.6e\n', ...
    LimitAmplitude);
fprintf('Nonzero zero-input samples = %d\n', ...
    length(LimitSamples));

fprintf('Overflow samples = %d\n',OverflowCount);
fprintf('Maximum absolute output = %.6e\n',MaxOverflow);
```

```
========== QUANTIZATION RESULTS ==========
Coefficient word length = 8 bits
Product word length = 16 bits
Maximum coefficient/output error = 4.405764e-03
Coefficient SQNR = 32.23 dB
Maximum product quantization error = 1.651951e-01
Product SQNR = 1.57 dB
Zero-input maximum amplitude = 9.216590e-03
Nonzero zero-input samples = 177
Overflow samples = 194
Maximum absolute output = 1.359910e+00
```

7. RESULT TABLE

```
Test = ["Coefficient Quantization";
        "Product Quantization";
        "Zero-Input Test";
        "Overflow Test"];

MaximumValue = [MaxCoeffError;
                MaxProdError;
                LimitAmplitude;
                MaxOverflow];

SQNR_dB = [SQNR_coeff;
           SQNR_prod;
           NaN;
           NaN];

T = table(Test,MaximumValue,SQNR_dB);
disp(T);
```

Test	MaximumValue	SQNR_dB
"Coefficient Quantization"	0.0044058	32.233
"Product Quantization"	0.1652	1.5673
"Zero-Input Test"	0.0092166	NaN
"Overflow Test"	1.3599	NaN

8. QUANTIZED COEFFICIENTS

```
fprintf('\nOriginal numerator coefficients:\n');
disp(b);

fprintf('Quantized numerator coefficients:\n');
disp(bq);

fprintf('Original denominator coefficients:\n');
disp(a);

fprintf('Quantized denominator coefficients:\n');
disp(aq);
```

```
Original numerator coefficients:
    0.0048    0.0193    0.0289    0.0193    0.0048

Quantized numerator coefficients:
    0.0079    0.0157    0.0315    0.0157    0.0079

Original denominator coefficients:
    1.0000   -2.3695    2.3140   -1.0547    0.1874

Quantized denominator coefficients:
    1.0000   -2.3701    2.3150   -1.0551    0.1890
```

9. FILTER STABILITY CHECK

```
p_ideal = roots(a);
p_quantized = roots(aq);

fprintf('\nMaximum ideal pole magnitude = %.6f\n', ...
    max(abs(p_ideal)));

fprintf('Maximum quantized pole magnitude = %.6f\n', ...
    max(abs(p_quantized)));

if all(abs(p_quantized)<1)
    fprintf('Quantized filter is stable.\n');
else
    fprintf('Quantized filter is unstable or marginally stable.\n');
end
```

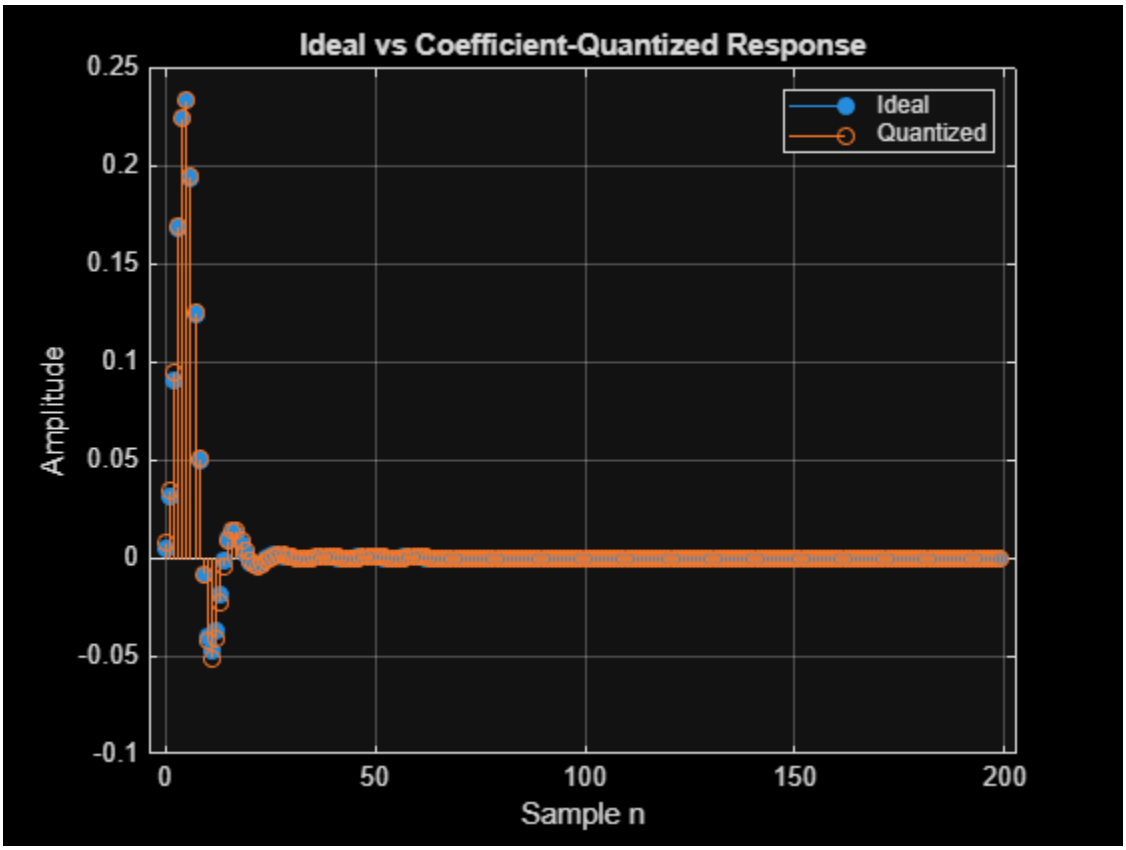
```
Maximum ideal pole magnitude = 0.795436
Maximum quantized pole magnitude = 0.797043
Quantized filter is stable.
```

10. PLOT: IDEAL VS QUANTIZED RESPONSE

```
figure('Name','Ideal vs Quantized Response');

stem(0:N-1,y_ideal,'filled');
hold on;
stem(0:N-1,y_coeff);

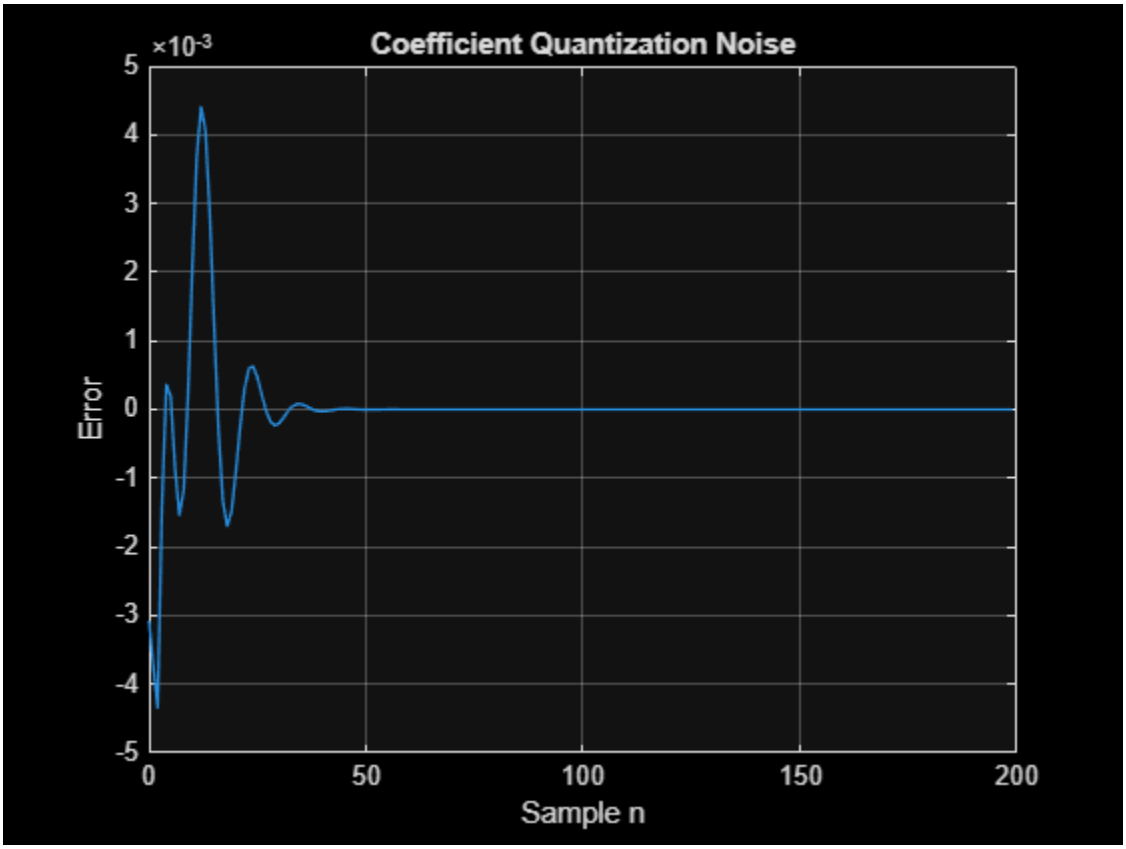
grid on;
xlabel('Sample n');
ylabel('Amplitude');
title('Ideal vs Coefficient-Quantized Response');
legend('Ideal','Quantized');
```



11. COEFFICIENT QUANTIZATION ERROR

```
figure('Name','Coefficient Quantization Noise');

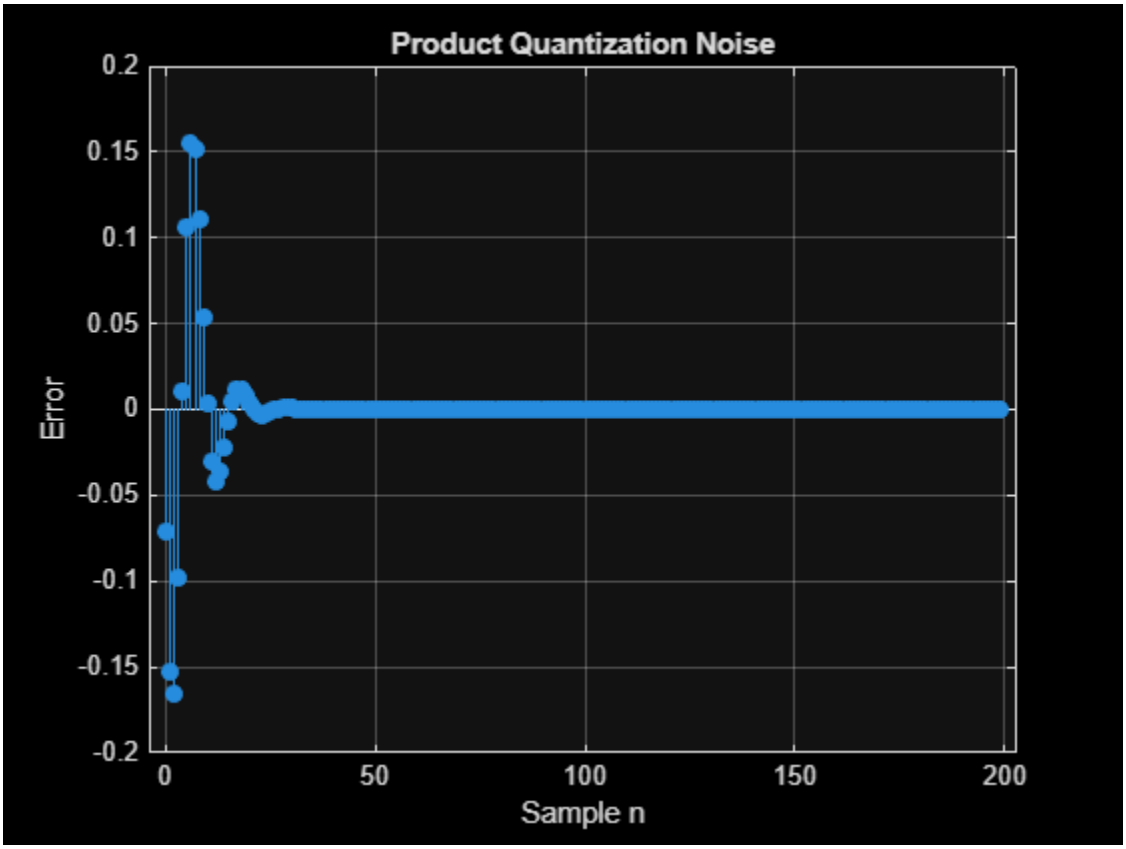
plot(0:N-1,e_coeff,'LineWidth',1);
grid on;
xlabel('Sample n');
ylabel('Error');
title('Coefficient Quantization Noise');
```



12. PRODUCT QUANTIZATION ERROR

```
figure('Name','Product Quantization Noise');

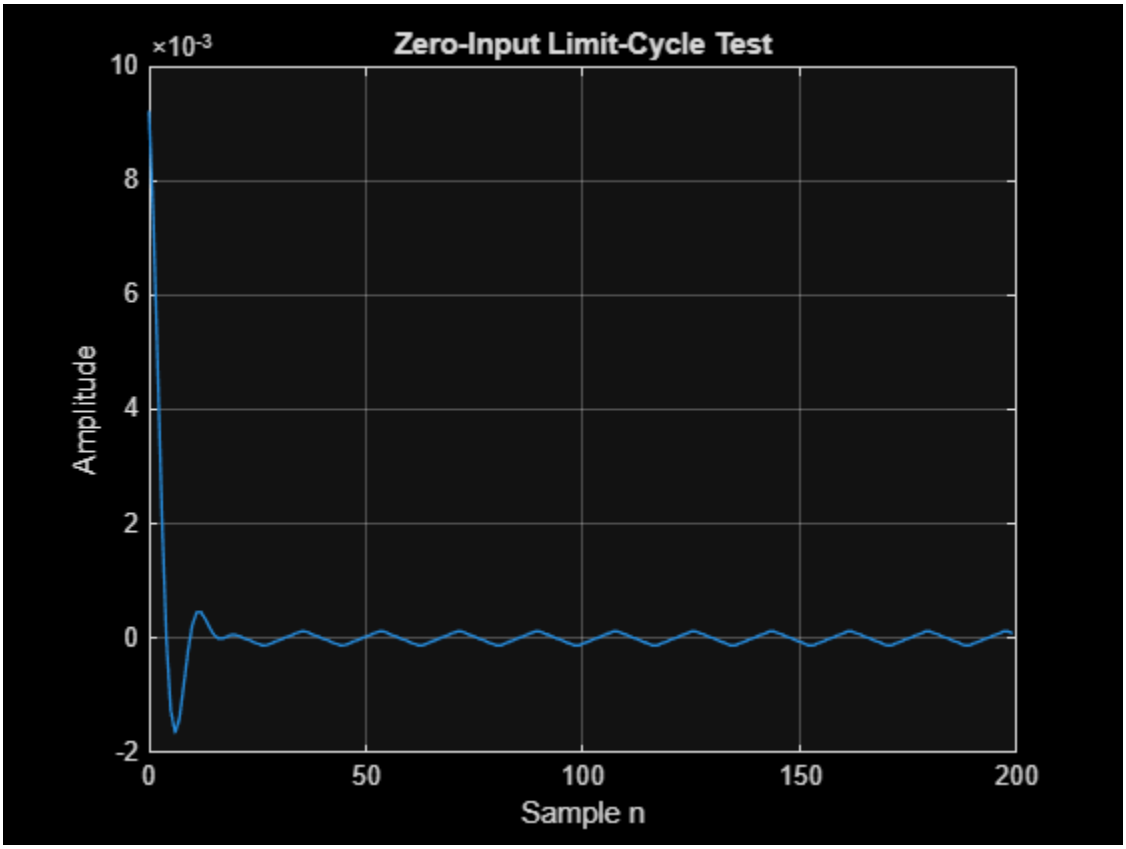
stem(0:N-1,y_coeff-y_prod,'filled');
grid on;
xlabel('Sample n');
ylabel('Error');
title('Product Quantization Noise');
```



13. ZERO-INPUT LIMIT-CYCLE TEST

```
figure('Name','Zero Input Limit Cycle');

plot(0:N-1,y_zero,'LineWidth',1);
grid on;
xlabel('Sample n');
ylabel('Amplitude');
title('Zero-Input Limit-Cycle Test');
```

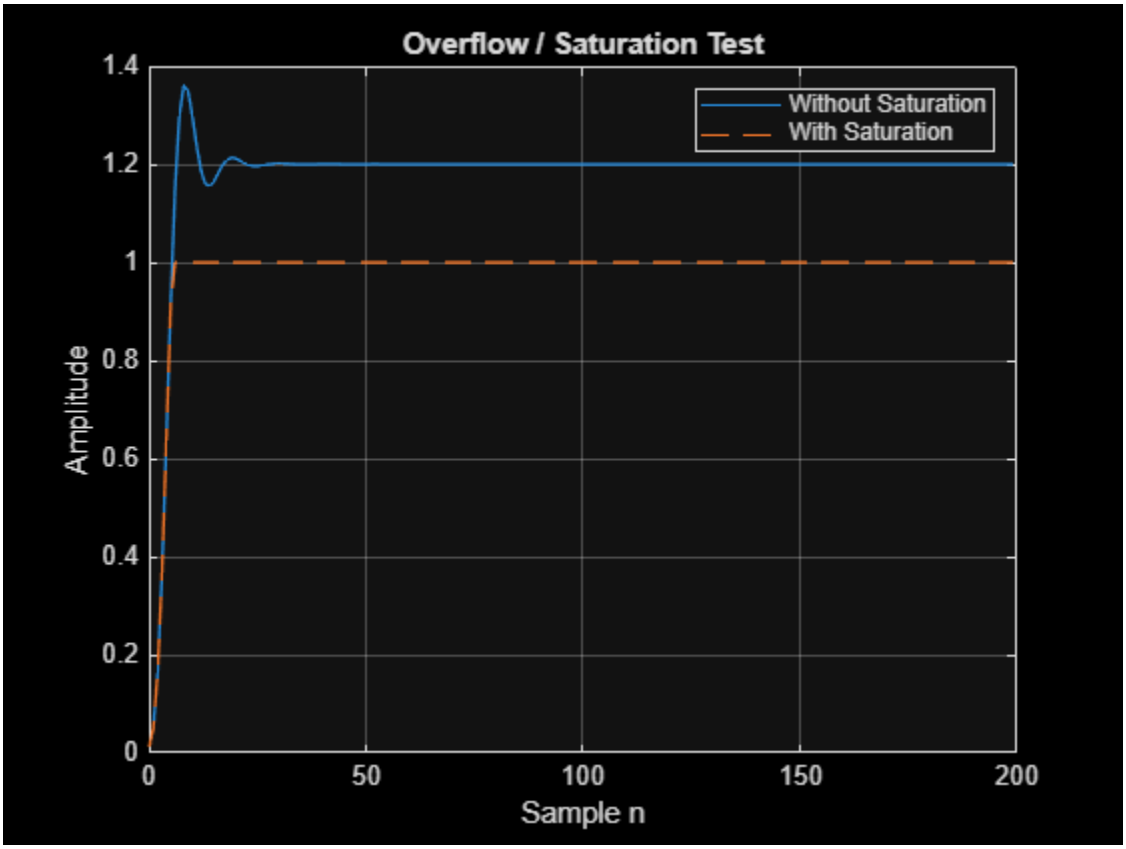


14. OVERFLOW / SATURATION

```
figure('Name','Overflow and Saturation');

plot(0:N-1,y_over);
hold on;
plot(0:N-1,y_sat,'--');

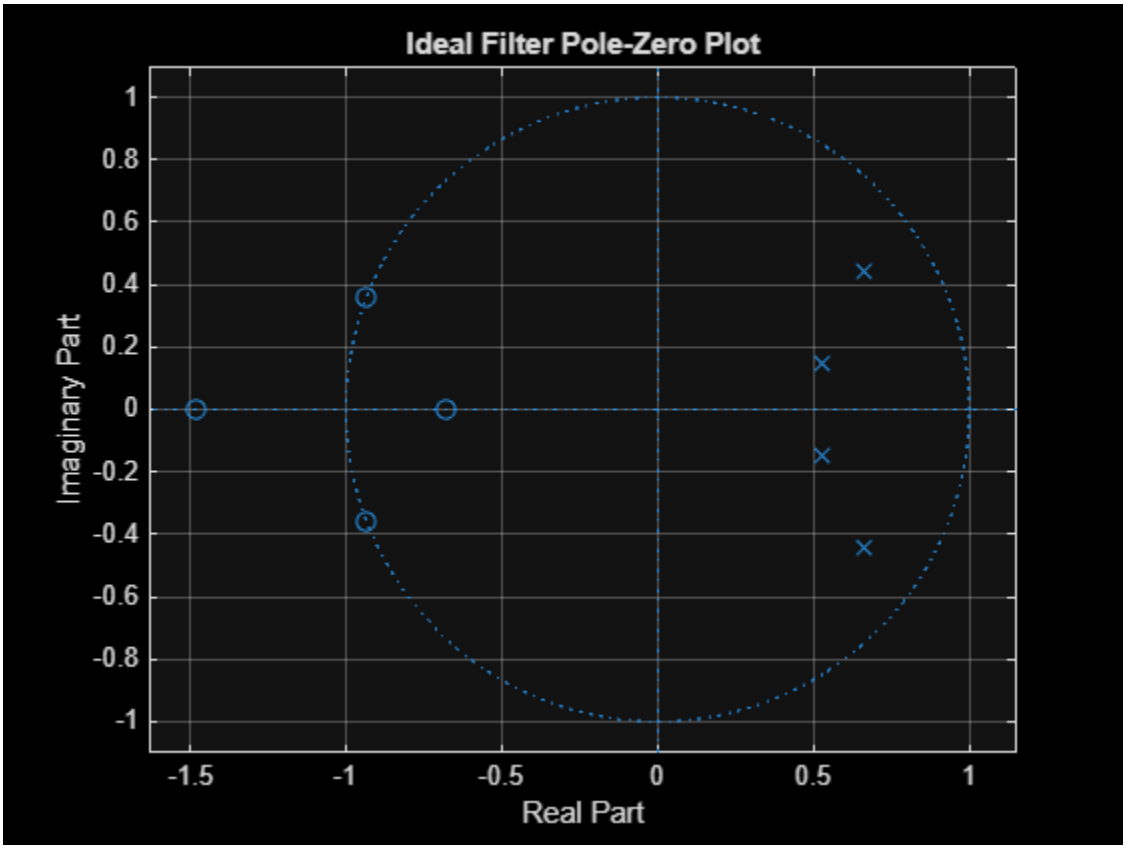
grid on;
xlabel('Sample n');
ylabel('Amplitude');
title('Overflow / Saturation Test');
legend('Without Saturation','With Saturation');
```



15. IDEAL POLE-ZERO PLOT

```
figure('Name','Ideal Filter Pole-Zero Plot');

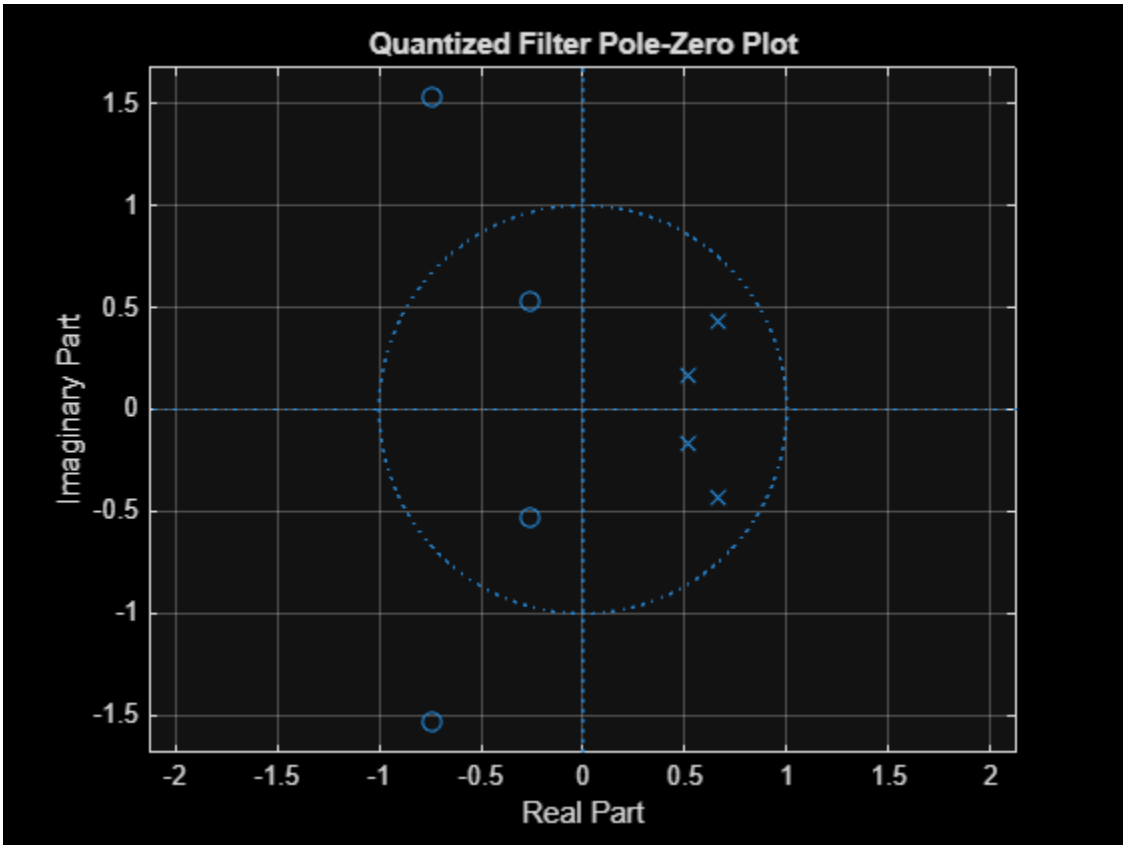
zplane(b,a);
grid on;
title('Ideal Filter Pole-Zero Plot');
```



16. QUANTIZED POLE-ZERO PLOT

```
figure('Name','Quantized Filter Pole-Zero Plot');

zplane(bq,aq);
grid on;
title('Quantized Filter Pole-Zero Plot');
```



Published with MATLAB® R2026b